

Individual Final Report

Group 3, G31615327 Jiatong Peng

1. Introduction. An overview of the project and an outline of the shared work.

This project aims to explore controllable human face generation using deep learning techniques. Specifically, we utilize the CelebA dataset to build a system that can generate facial images conditioned on user-specified attributes, such as hair color, gender, and accessories. In addition to generation, the project also incorporates an evaluation mechanism to assess whether the generated images accurately reflect the intended attributes.

The overall framework of the project consists of three main components. First, a Conditional Variational Autoencoder (cVAE) is employed to generate facial images based on input attribute vectors. Second, an attribute classifier is used as a surrogate model to evaluate the generated images by predicting their attributes. Third, an interactive web-based interface is developed using Streamlit to allow users to input desired attributes, generate images, and visualize evaluation results in real time.

The work for this project was carried out collaboratively within the group. The implementation of the cVAE training pipeline was primarily developed by Zachary Cardell, including the model architecture and training procedure. Irene Xia involved training and optimizing the attribute classifier used for evaluation. My contribution to the project focused on integrating the generation and evaluation pipeline, and developing the interactive Streamlit application for demonstration and deployment.

Component	Developer	Description
Multi-label Classifier (Train script.py)	Irene Xia	DenseNet169 + BiLSTM classifier for all 40 attributes; complete training pipeline including preprocessing, disk caching, SWA, threshold tuning, and output generation
CVAE (cVAE_train.py + cVAE_train_option1_sigma.py)	Zachary Cardell	Conditional VAE architecture, training loop, and image generation
Streamlit Demo (streamlit_demo.py)	Jiatong Peng (me)	Web interface integrating both models for interactive generation and evaluation
Report Writing	All members	Each wrote individual report; group report compiled jointly

2. Description of your individual work.

2.1 Overview of My Contribution

My individual contribution to this project focuses on debugging the cVAE model outputs, ensuring consistency between the generation and classification pipelines, and integrating the system into an interactive Streamlit application. Rather than developing new models, my work primarily involved identifying sources of errors, validating data processing steps, and improving the reliability of the end-to-end pipeline.

2.2 Debugging the cVAE Output

During the development process, the generated images from the cVAE model showed noticeable quality issues, such as unnatural color tones and blurry details. To identify the cause, I systematically examined different components of the pipeline, including model loading, preprocessing, and visualization.

The key issue was traced to the model loading step. The system was using a checkpoint corresponding to the best validation performance rather than the final trained model. This mismatch resulted in degraded visual quality of generated images. To resolve this, I modified the training and loading logic to ensure that the correct final model was saved and used for inference.

After this correction, the visual quality of generated images improved significantly, confirming that the issue was not due to the model architecture itself but rather due to incorrect model selection.

2.3 Ensuring Consistent Preprocessing

Another important part of my work was verifying the consistency of preprocessing between the classifier training pipeline and the demo inference pipeline.

The classifier was trained on images that had undergone a specific sequence of preprocessing steps, including contrast enhancement using CLAHE, Gaussian blurring, resizing, and normalization. I carefully checked whether the same steps were applied to the generated images before passing them to the classifier in the demo.

After reviewing the code, I confirmed that the preprocessing steps in the demo were consistent with those used during training. Therefore, the misclassification of certain attributes (e.g., eyeglasses) is unlikely to be caused by preprocessing mismatch. Instead, it is more likely due to the domain gap between generated images and real images in the dataset.

2.4 Integration of Generation and Evaluation Pipeline

I also worked on integrating the cVAE generation module with the attribute classifier to form a complete pipeline.

In this pipeline, user-selected attributes are first converted into an input vector for the cVAE model, which generates corresponding facial images. These images are then processed and passed into the classifier to predict attribute probabilities. The predictions are compared with the target attributes to evaluate how well the generated images match the intended conditions.

This integration required careful handling of image resizing, normalization, and threshold-based decision rules to ensure that the classifier outputs were meaningful and comparable.

2.5 Streamlit Application Development

Finally, I contributed to the development of the Streamlit-based user interface for the project. The application allows users to select facial attributes, generate images using the cVAE model, and view classifier-based evaluation results.

I implemented features such as attribute selection, model selection, and result visualization. I also configured the application for external access using ngrok, enabling real-time demonstration and testing.

2.6 Experimental Exploration Using the Streamlit Interface

In addition to system integration, I used the Streamlit interface as a tool to explore the behavior of the overall pipeline. By selecting different combinations of facial attributes, I generated images and observed the corresponding classifier outputs.

Through this process, I systematically tested a variety of attribute combinations and identified patterns in which attributes were more likely to fail in generation or classification. This helped me better understand the strengths and limitations of the current system and ensured that the pipeline could be reliably used for further analysis and demonstration.

3. Describe the portion of the work that you did on the project in detail. It can be figures, codes, explanation, pre-processing, training, etc.

3.1 Debugging and Revising the cVAE Model Saving Logic

During the development process, I observed that the generated images from the cVAE model sometimes showed unnatural color tones, such as a yellowish appearance, as well as blurry details. This raised concerns about whether the model itself was not properly trained or whether there was an issue in the inference pipeline.

To better understand the problem, I examined the generated images saved during training at different epochs. In particular, I compared images from early epochs with those from the final

epochs. I found that the images generated in the later stages of training did not exhibit the same yellowish artifacts and were visually more stable. This observation suggested that the model itself was capable of producing reasonable outputs, and the issue might be related to which model checkpoint was being used for inference.

After further investigation, I identified that the original training script only saved the model corresponding to the best validation loss during training. In the original version, the training function ended immediately after printing "Training complete." and returning the model, without explicitly saving the final trained model.

The original ending was:

```
print("Training complete.")  
return model
```

However, for generative models such as cVAE, the model with the lowest validation loss does not necessarily produce the best visual results. Based on the earlier observation that later-epoch outputs were more visually stable, I modified the training script to explicitly save the final model after all epochs were completed.

The following code was added at the end of the `train()` function in `cVAE_train_revised.py`, right before "Training complete.":

```
torch.save(  
    model.state_dict(),  
    os.path.join(MODELS_DIR, f"model_{NICKNAME}_final.pt")  
)  
print(f"    → final model saved to model_{NICKNAME}_final.pt")  
  
print("Training complete.")  
return model
```

This modification creates an additional file named:

`model_cVAE_Group3_final.pt`

The key idea of this change was to separate the best validation checkpoint from the final trained model. The original file `model_cVAE_Group3.pt` corresponds to the best validation-loss model, while the new file represents the model after the full training process.

After applying this modification, I updated the demo to load the final model instead of the validation-based checkpoint. This resolved the earlier issue with yellowish outputs and improved the overall visual quality of the generated images.

In addition, I integrated an updated cVAE model provided by my teammate into the demo.

This required adapting the model loading pipeline to support different architectures, including resolving compatibility issues between training scripts and inference code.

3.2 Verification of Preprocessing Consistency

In this project, I reused the preprocessing pipeline from Irene's classifier and verified that the same steps were applied in the Streamlit demo.

From the classifier training script, images are processed using CLAHE for contrast enhancement, followed by Gaussian blur, resizing, and normalization before being fed into the model.

I then checked the demo code and confirmed that generated images go through the same sequence of preprocessing steps before classification, including CLAHE, Gaussian blur, resizing to 224×224, and ImageNet normalization.

This verification ensured that the classifier receives inputs with a distribution consistent with its training data.

3.3 Integration of Generation and Classification in the Demo

I integrated the cVAE generation module and Irene's classifier into the Streamlit demo so that the system could generate images and evaluate selected attributes in one workflow.

In the demo, selected attributes are first converted into a 40-dimensional attribute vector following the CelebA attribute order. This vector is then passed into the cVAE model to generate facial images:

```
attr_vec = torch.zeros(len(CLASS_NAMES), device=device)
```

```
for attr, val in target_attrs.items():
    if attr in CLASS_NAMES:
        attr_vec[CLASS_NAMES.index(attr)] = float(val)
```

```
attr_vec = attr_vec.unsqueeze(0).repeat(n_images, 1)
```

```
with torch.no_grad():
    imgs = cvae_model.generate(attr_vec)
```

After generation, the images are passed through the classifier preprocessing function and then evaluated by the trained classifier:

```
x_clf = preprocess_generated_for_classifier(imgs)
```

```
with torch.no_grad():
```

```
logits = clf(x_clf)
probs = torch.sigmoid(logits)
preds = (probs >= thresholds).int().cpu().numpy()
```

The demo then summarizes the classifier results for the selected target attributes, including the average classifier probability, predicted positive rate, and whether the prediction matches the selected condition. This integration made it possible to test generation and classification together in one interface.

This integration was further extended to support multiple cVAE models, allowing dynamic selection between different trained models within the demo.

3.4 Streamlit Application Development

3.4.1 Conflict Design Motivation

A key part of the Streamlit demo design was handling attribute combinations that may lead to unrealistic or unreliable results. The idea of introducing conflict detection originated from two sources.

First, from domain intuition, some attribute combinations in the CelebA dataset are logically inconsistent. For example, attributes such as Bald and Bangs, or No_Beard and Mustache, rarely co-exist in real-world images.

Second, after reviewing Irene's classifier results and testing the demo with different attribute combinations, I observed that certain combinations tended to produce unstable generation or inconsistent classification outputs. This motivated me to incorporate a conflict-handling mechanism to improve both usability and interpretability.

3.4.2 Iterative Refinement of Conflict Rules

The definition of conflict combinations was developed iteratively rather than fixed from the beginning.

In the initial stage, I defined a set of hard conflicts, which represent logically contradictory attribute pairs. These include combinations such as:

```
("Bald", "Bangs"),
("No_Beard", "Mustache"),
("Black_Hair", "Blond_Hair")
```

These rules were implemented as explicit attribute pairs in the demo.

As the project progressed, I expanded this design by introducing soft conflicts. These correspond to combinations that are not impossible but are uncommon or less reliably handled by the model, such as:

Male with heavy makeup
Female with facial hair
Female with bald

This refinement was guided by both dataset intuition and observations from classifier behavior during testing. The distinction between hard and soft conflicts allows the system to provide more informative feedback without overly restricting user input.

3.4.3 Conflict Detection Mechanism

To support this design, I implemented a conflict detection function that analyzes the selected attributes and identifies potential issues.

```
def detect_conflicts(target_attrs):
    hard_conflicts = []
    soft_conflicts = []

    selected_set = set(target_attrs.keys())

    for a, b in HARD_CONFLICT_PAIRS:
        if a in selected_set and b in selected_set:
            hard_conflicts.append((
                DISPLAY_NAME.get(a, a),
                DISPLAY_NAME.get(b, b),
                ATTRIBUTE_TO_GROUP.get(b) or ATTRIBUTE_TO_GROUP.get(a) or "General"
            ))

    # Soft conflict: Male + makeup/lipstick
    if target_attrs.get("Male") == 1:
        for attr in ["Heavy_Makeup", "Wearing_Lipstick"]:
            if target_attrs.get(attr) == 1:
                soft_conflicts.append((
                    "Male",
                    DISPLAY_NAME.get(attr, attr),
                    ATTRIBUTE_TO_GROUP.get(attr, "Makeup & Skin")
                ))

    # Soft conflict: Female + facial hair
    if target_attrs.get("Male") == 0:
        for attr in ["Mustache", "Goatee", "Sideburns", "5_o_Clock_Shadow"]:
```

```

        if target_attrs.get(attr) == 1:
            soft_conflicts.append((
                "Female",
                DISPLAY_NAME.get(attr, attr),
                "Facial Hair"
            ))

    # Soft conflict: Female + Bald
    if target_attrs.get("Male") == 0 and target_attrs.get("Bald") == 1:
        soft_conflicts.append(("Female", "Bald", "Hair"))

    return hard_conflicts, soft_conflicts

```

This function checks the selected attribute set against predefined rules and returns two categories of results: hard conflicts and soft conflicts. These outputs are then used by the interface to generate appropriate feedback messages.

3.4.4 UI Warning System Design

Based on the detected conflicts, I designed a warning system in the user interface to provide real-time feedback.

Hard conflicts are displayed as error messages indicating strong contradictions
 Soft conflicts are displayed as warnings indicating unusual but allowed combinations

The messages are dynamically rendered in the interface using grouped containers:

```

if msg_type == "error":
    conflict_boxes[group_name].error(msg)
else:
    conflict_boxes[group_name].warning(msg)

```

This design choice was intentional. Instead of blocking conflicting inputs, the system allows users to proceed while clearly indicating potential issues. This supports exploratory use of the demo, where users can test how the model behaves under challenging or uncommon conditions.

3.4.5 Model Selection Implementation

To support comparison between different cVAE models, I added a model selection function in the Streamlit demo. This was necessary because the original and updated cVAE models were trained using different architectures, so they could not be loaded using the same CVAE class.

First, I imported the old and updated model architectures separately:

```

from cVAE_train_revised import CVAE as CVAE_old

```



```
from cVAE_model_option1_sigma import CVAE as CVAE_new1
```

Then I created a dictionary to store each model path and its corresponding model type:

```
CVAE_MODEL_OPTIONS = {
    "Final Model": {
        "path": "/home/ubuntu/Final-Project-Group3/Models/model_cVAE_Group3_final.pt",
        "type": "new1"
    },
    "Old Model": {
        "path":
"/home/ubuntu/Final-Project-Group3/Models/model_cVAE_Group3_previous.pt",
        "type": "old"
    },
}
```

The loading function then selects the correct CVAE class based on the model type:

```
@st.cache_resource
def load_cvae(model_path, model_type):
    if model_type == "old":
        model = CVAE_old().to(device)
    elif model_type == "new1":
        model = CVAE_new1().to(device)
    else:
        raise ValueError(f"Unknown model_type: {model_type}")

    model.load_state_dict(torch.load(model_path, map_location=device))
    model.eval()
    return model
```

In the interface, I used a select box to allow users to choose which model to use. The selected model path and model type were stored in session state and later used during generation:

```
model_choice = st.selectbox(
    "Choose a cVAE model",
    list(CVAE_MODEL_OPTIONS.keys())
)

st.session_state.cvae_model_choice = model_choice
st.session_state.cvae_model_path = CVAE_MODEL_OPTIONS[model_choice]["path"]
st.session_state.cvae_model_type = CVAE_MODEL_OPTIONS[model_choice]["type"]
```

This implementation made it possible to switch between the old and updated cVAE models

without changing the rest of the generation and evaluation pipeline.

3.4.6 Upload Image Classification Implementation

I also implemented the upload image classification module in the Streamlit demo. This module allows users to upload a local face image and classify its CelebA attributes using Irene's trained classifier.

```
uploaded_file = st.file_uploader(
    "Upload a JPEG or PNG image",
    type=["jpg", "jpeg", "png"]
)
```

Before running the classifier, I added a basic face detection step to avoid applying the face attribute classifier to non-face images:

```
def detect_face(uploaded_file):
    uploaded_file.seek(0)

    file_bytes = np.asarray(bytearray(uploaded_file.read()), dtype=np.uint8)
    img = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    face_cascade = cv2.CascadeClassifier(
        cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
    )

    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    return len(faces) > 0
```

If a face is detected, the uploaded image is passed through the same preprocessing function used for generated images and then classified:

```
def classify_uploaded_image(uploaded_file):
    image = Image.open(uploaded_file).convert("RGB")

    x = preprocess_for_classifier_pil(image)

    with torch.no_grad():
        logits = clf(x)
        probs = torch.sigmoid(logits).squeeze(0)
        preds = (probs >= thresholds).int()

    rows = []
    for i, attr in enumerate(CLASS_NAMES):
```

```

        if preds[i].item() == 1:
            rows.append({
                "attribute": attr,
                "probability": round(float(probs[i].item()), 3),
                "prediction": 1
            })

    result_df = pd.DataFrame(rows).sort_values(
        "probability",
        ascending=False
    ).reset_index(drop=True)

    return image, result_df

```

The positive predictions are collected into a table and sorted by probability.

If no face is detected, the interface shows a warning instead of producing meaningless classifier outputs:

```

if detect_face(uploaded_file):
    go_to("upload_result")
else:
    st.warning(
        "No face detected. This classifier is trained on face images only. "
        "Please upload a clear face image."
    )

```

This upload module extends the demo beyond generated images and allows the classifier to be tested on external images, while the face detection warning improves the reliability of the interface.

3.4.7 Summary

Overall, this component integrates rule-based design with empirical observations: conflict definitions are based on both logical constraints and model behavior, the rules were refined iteratively through testing, and the UI provides informative feedback without restricting user interaction. This design improves both the usability of the demo and the ability to explore the limitations of the model.

4. Results.

4.1 Experimental Setup

The experiments were conducted using the Streamlit demo developed in this project. The

interface allows users to select facial attributes, generate images using the cVAE model, and evaluate the generated results using the trained classifier.

All results in this section are based on the currently available model versions (may not be the final submitted models) at the time of submission. The purpose of these experiments is to analyze the behavior of the system under different input conditions rather than to achieve optimal performance.

4.2 Generation with Valid Attribute Combinations

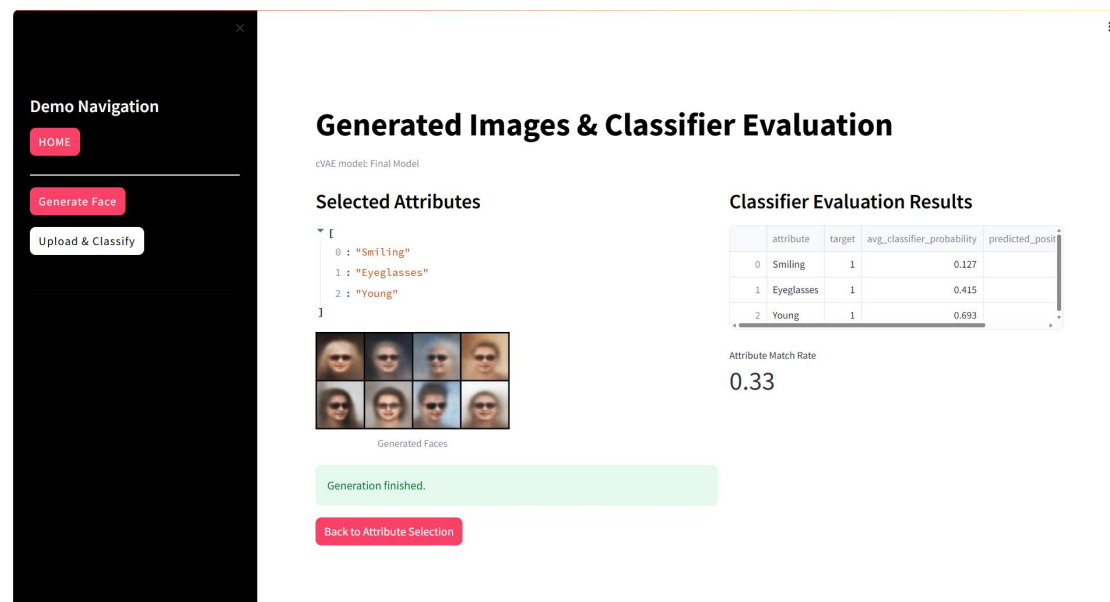


Figure 1

To evaluate whether the system can correctly generate images under normal conditions, I tested a valid attribute combination consisting of Smiling, Eyeglasses, and Young.

From visual inspection, the generated faces appear generally coherent. The Young attribute is clearly reflected in most samples. Both Eyeglasses and Smiling also appear to be reasonably well generated: glasses are visible in several images, and most faces show clear smiling expressions.

However, the classifier results show a different pattern. As summarized in Table 2:

Table 2

Attribute	Target	Avg Classifier Probability	Predicted Positive Rate	Match
Smiling	1	0.127	0.000	No
Eyeglasses	1	0.415	0.375	No
Young	1	0.693	1.000	Yes

Young achieves a relatively high average probability (0.693) and is correctly predicted. Eyeglasses, despite being visually present, has only a moderate average probability (0.415) and a low predicted positive rate (0.375).

Smiling, although clearly visible to the human eye, shows a very low average probability (0.127).

and is not correctly predicted

The overall attribute match rate is 0.33, indicating that only one of the three selected attributes is reliably captured by the classifier.

This comparison highlights a gap between human perception and classifier evaluation. Even when attributes such as Smiling and Eyeglasses appear visually convincing, the classifier may fail to recognize them consistently. This suggests that the generated images, while plausible to humans, may not fully align with the statistical patterns learned by the classifier from real images.

4.3 Comparison Between Valid and Conflicting Attribute Combinations

To evaluate the robustness of the system under more challenging conditions, I tested several conflicting attribute combinations using the Streamlit interface.

The selection of conflict cases follows the conflict design introduced in Section 3.4, including both hard conflicts and soft conflicts:

Hard conflicts: attribute pairs that are logically inconsistent

e.g., Bald + Bangs, No_Beard + Mustache

Soft conflicts: attribute combinations that are uncommon or underrepresented in the dataset

e.g., Female + Mustache, Male + Heavy_Makeup

For each combination, I generated multiple images using the cVAE model and evaluated them using the classifier. Both the visual appearance of the generated images and the classifier outputs were recorded for comparison with the valid attribute combinations in Section 4.2.

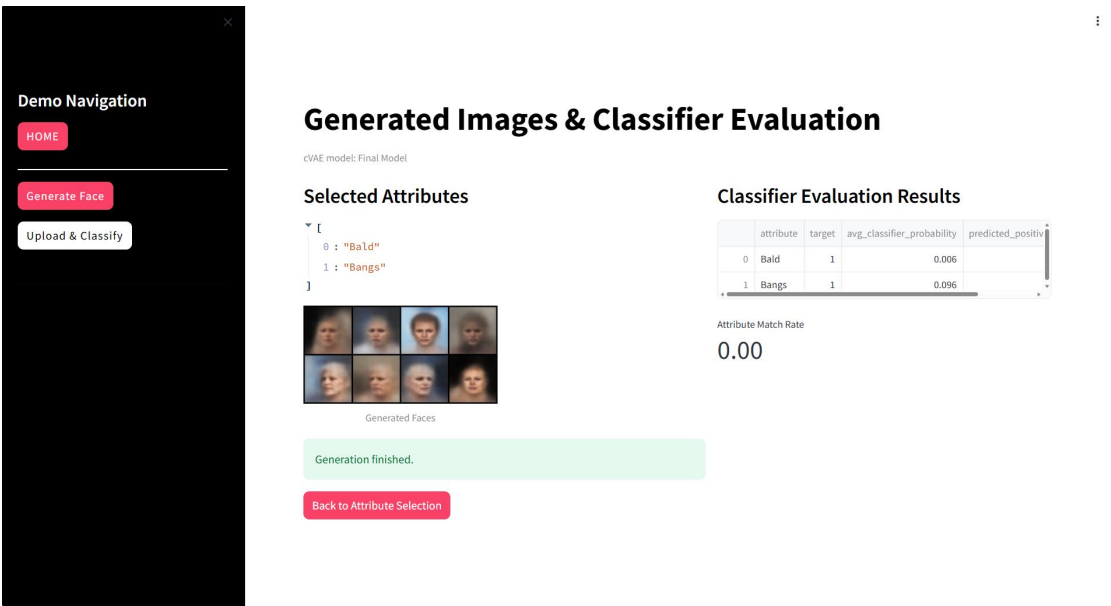


Figure 3

Figure 3 shows the generated images and classifier evaluation results for the hard conflict combination Bald + Bangs.

From visual inspection, the generated faces are noticeably less stable compared to the valid combinations in Section 4.2. The images appear blurrier and less well-defined, and the hair-related attributes are not clearly expressed. In most cases, neither Bald nor Bangs is convincingly represented. This reflects the inherent contradiction of the input condition, as these two attributes are mutually exclusive in real-world data.

The classifier results further confirm this failure. As summarized in Table 4:

Table 4

Attribute	Target	Avg Classifier Probability	Predicted Positive Rate	Match
Bald	1	0.006	0.000	No
Bangs	1	0.096	0.000	No

The overall attribute match rate is 0.00, indicating that neither attribute is successfully recognized by the classifier.

Compared with the valid combinations in Section 4.2, this case demonstrates a clear degradation in both generation quality and classification performance. Unlike the earlier case, where attributes were visually present but sometimes misclassified, here both the human perception and classifier evaluation indicate failure.

This result highlights the limitation of the model when handling logically inconsistent conditions. Since such combinations are not represented in the training data, the model is unable to generate meaningful outputs, leading to unstable images and unreliable attribute predictions.

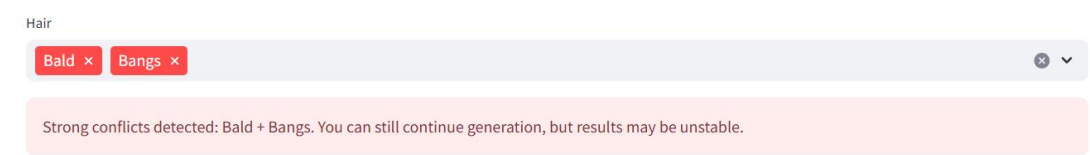


Figure 5

This figure 5 also confirms that the conflict detection and warning system designed in Section 3.4 is functioning correctly, as the interface automatically identifies this combination as a strong conflict and alerts the user before generation.

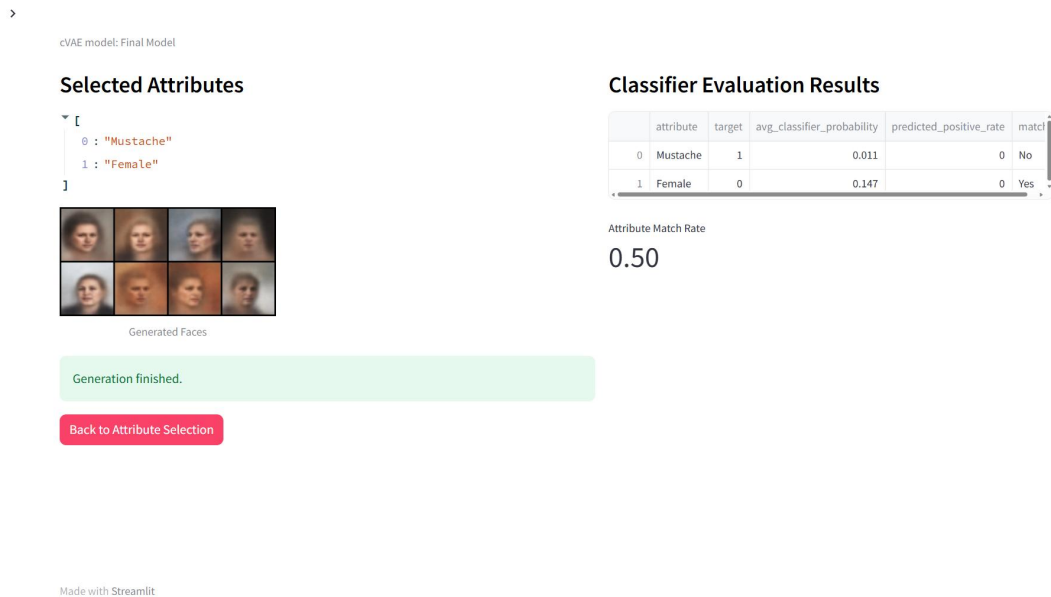


Figure 6

Figure 6 shows the generated images and classifier evaluation results for the soft conflict combination Female + Mustache.

From visual inspection, the generated faces clearly exhibit female characteristics, such as facial structure and overall appearance. However, the Mustache attribute is not visible in most of the generated images. This suggests that when given a soft conflict, the model tends to prioritize one attribute over the other rather than attempting to represent both.

The classifier results are consistent with this observation. As summarized in Table 7:

Table 7

Attribute	Target	Avg Classifier Probability	Predicted Positive Rate	Match
Mustache	1	0.011	0.000	No
Female	0	0.147	0.000	Yes

The overall attribute match rate is 0.50, indicating that only one of the two attributes is successfully recognized.

Compared with the hard conflict case (Bald + Bangs), where both attributes fail, this soft conflict shows a different behavior. The model does not collapse completely but instead selectively represents the more dominant or common attribute (Female) while ignoring the conflicting one (Mustache).

This result suggests that for soft conflicts, the model exhibits a bias toward more frequent or visually dominant attributes in the training data, rather than producing unstable outputs.

Gender

Female

Age

Skip

Hair

Choose an option

Face Structure

Choose an option

Facial Hair

Mustache x

Unusual combinations detected: Female + Mustache. These are rare in the dataset, but generation is still allowed.

Figure 8

In addition, the interface correctly identifies this combination as a soft conflict and displays a warning message (“Unusual combinations detected: Female + Mustache”), confirming that the warning mechanism for soft conflicts is functioning as intended.

4.4 Comparison Between Old and Updated cVAE Models

To further evaluate the impact of model architecture on generation performance, I compared the original cVAE model with an updated version provided by my teammate using the same attribute combination (Smiling, Eyeglasses, and Young).

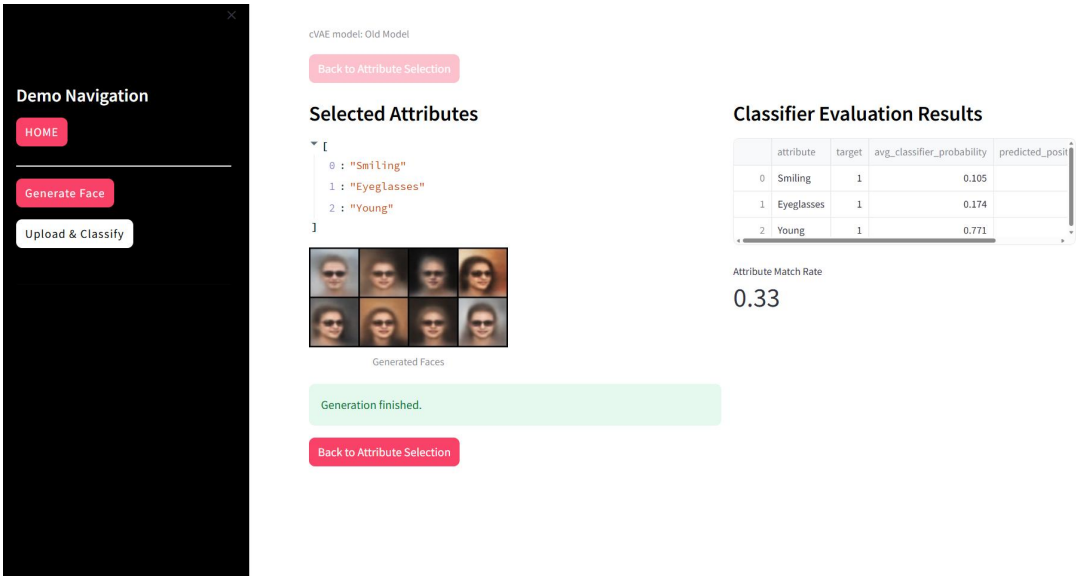


Figure 9

Table 10 summarizes the classifier evaluation results for the original model.

Table 10

Attribute	Target	Avg Classifier Probability	Predicted Positive Rate	Match
Smiling	1	0.105	0.000	No
Eyeglasses	1	0.174	0.125	No
Young	1	0.771	1.000	Yes

The results show that only the attribute "Young" is reliably captured by the original model. Both "Smiling" and "Eyeglasses" have very low classifier probabilities and are not correctly recognized. The overall attribute match rate is 0.33.

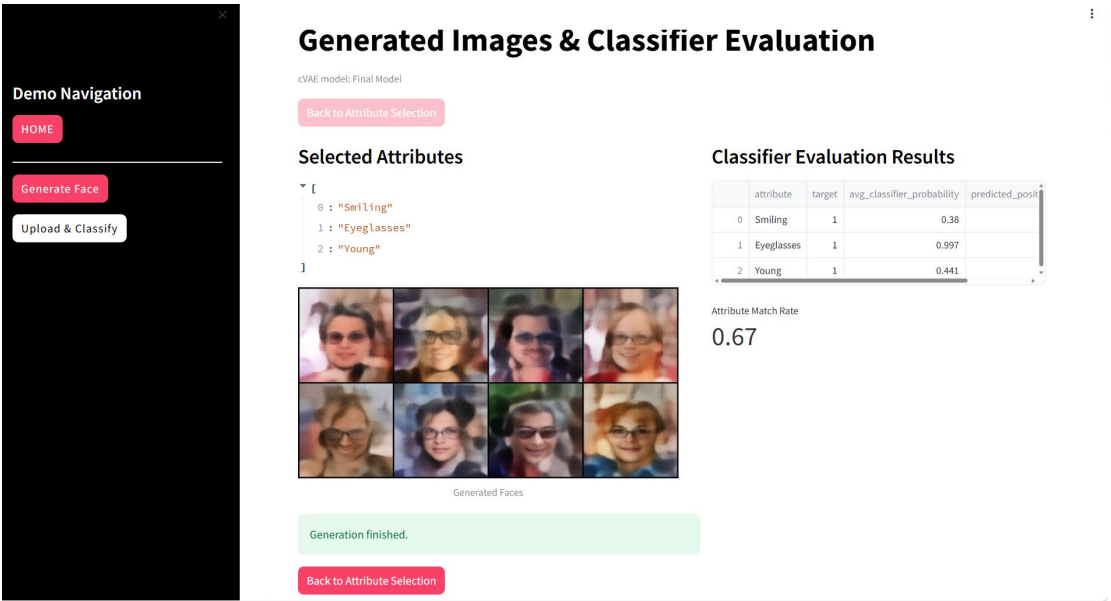


Figure 11

Table 12 presents the results for the updated model.

Table 12

Attribute	Target	Avg Classifier Probability	Predicted Positive Rate	Match
Smiling	1	0.380	0.500	Yes
Eyeglasses	1	0.997	1.000	Yes
Young	1	0.441	0.375	No

Compared with the original model, the updated model shows a clear improvement in capturing certain attributes. Both "Smiling" and "Eyeglasses" are successfully detected, with significantly higher classifier probabilities. As a result, the overall attribute match rate increases from 0.33 to 0.67.

From visual inspection, both models generate recognizable facial structures, but the updated model produces more consistent representations of visually salient attributes. In particular, eyeglasses appear clearly in most generated images, and smiling expressions are more noticeable compared to the original model. However, the "Young" attribute becomes less stable in the updated model, indicating that improvements are not uniform across all attributes.

Overall, the updated model demonstrates improved performance in capturing visually distinctive features, leading to a higher attribute match rate. However, this improvement is attribute-dependent, suggesting that different model architectures may prioritize different

aspects of the data distribution.

4.5 Results on Uploaded Face Images

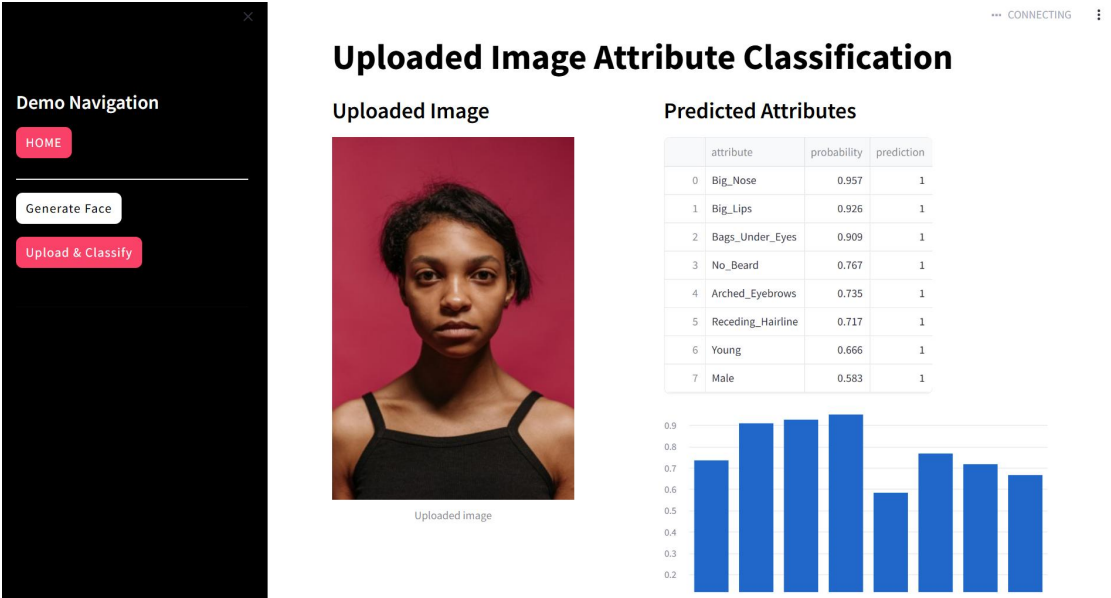


Figure 13

Figure 13 shows the classifier predictions for an uploaded face image selected from a royalty-free source.

From visual inspection, the image clearly contains a female face with no visible facial hair, relatively young appearance, and neutral expression. The facial features such as eyes, lips, and overall structure are clearly defined.

The classifier results are largely consistent with these observations:

Table 14

Attribute	Probability	Prediction
Big_Nose	0.957	1
Big_Lips	0.926	1
Bags_Under_Eyes	0.909	1
No_Beard	0.767	1
Arched_Eyebrows	0.735	1
Receding_Hairline	0.717	1
Young	0.666	1
Male	0.583	1

The classifier correctly identifies several attributes such as No_Beard and Young, which align well with human perception. However, there are also some noticeable inconsistencies. For example, the classifier predicts Male with a probability of 0.583, which contradicts the visual appearance of the subject. Similarly, attributes such as Receding_Hairline and Bags_Under_Eyes are predicted

with relatively high confidence, although they are not clearly evident.

Overall, this result suggests that the classifier performs reasonably well on real face images but still exhibits bias or uncertainty in certain attributes. While the model captures general facial characteristics, it may misclassify attributes that are more subjective or less visually prominent.

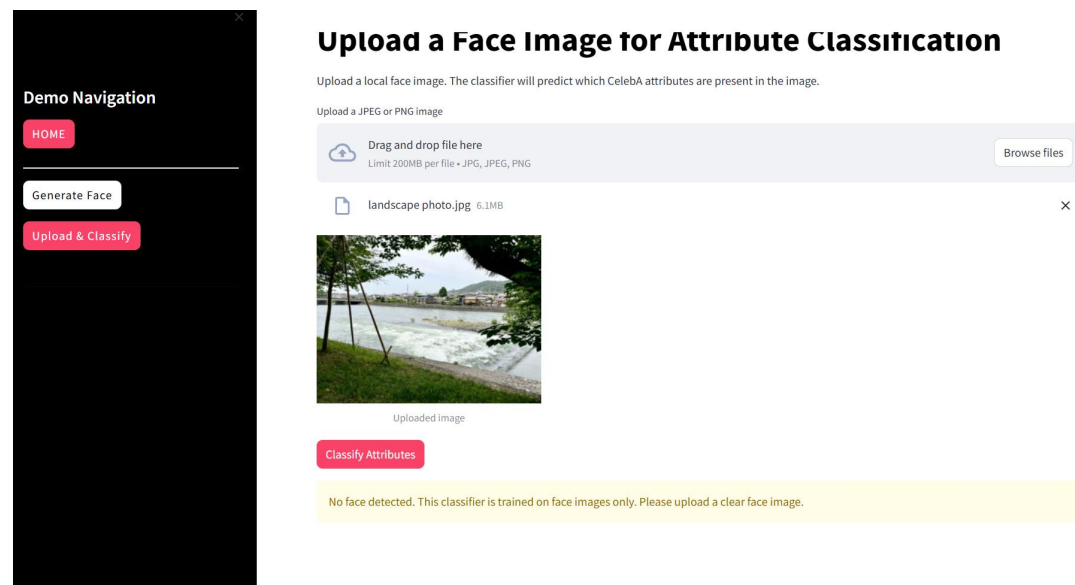


Figure 15

In addition, the system correctly detects that no face is present in the uploaded image and displays a warning message, indicating that the classifier is designed to operate on face images and that the input validation mechanism is functioning as intended.

5. Summary and conclusions.

In this project, I explored a face generation and evaluation system based on a cVAE model and an attribute classifier. The experiments show that the system is able to generate reasonable facial images under normal attribute combinations, although not all attributes are consistently captured.

When conflicting attribute combinations are used, the system exhibits clear limitations. For hard conflicts, both generation quality and classification performance degrade significantly. For soft conflicts, the model tends to prioritize more dominant attributes while ignoring others.

The classifier performs reasonably well on real face images but may produce incorrect predictions for certain attributes. When tested on non-face images, the system is able to detect invalid inputs and provide warning messages, which improves robustness.

Through this project, I gained a better understanding of the challenges in generative models, especially the gap between visual quality and quantitative evaluation, as well as the importance

of consistent preprocessing and system integration.

In future work, the system could be improved by using more advanced generative models, improving attribute disentanglement, and incorporating more reliable mechanisms for handling out-of-distribution inputs.

6. Calculate the percentage of the code that you found or copied from the internet.

Table 16

Code category	What it includes	Estimated lines
Copied / borrowed / AI-generated	cVAE training script, Irene's classifier, most Streamlit base UI/navigation/loading/evaluation structure	320
Modified from existing code	cVAE final model saving, switching demo to load final/old model options, preprocessing alignment, small integration adjustments	45
Added by me	Conflict combinations, hard/soft conflict logic, warning/error UI logic, selected attribute grouping/display adjustments that you explicitly directed	65

$$\begin{aligned}
 \text{Percentage} &= \frac{320 - 45}{320 + 65} \times 100 \\
 &= \frac{275}{385} \times 100 \approx 71.4\%
 \end{aligned}$$

7. References.

Liu, Z., Luo, P., Wang, X., & Tang, X. (2015). Deep learning face attributes in the wild. Proceedings of the IEEE International Conference on Computer Vision (ICCV).

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. International Conference on Learning Representations (ICLR).

PyTorch Team. (n.d.). PyTorch documentation. <https://pytorch.org/>

Streamlit Inc. (n.d.). Streamlit documentation. <https://streamlit.io/>

Pexels. (n.d.). Free stock photos. <https://www.pexels.com/>